

Software Engineering (CS24309) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PROJECT METRICS BANK

Software Engineering (CS24309)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

Complete Course Structure & Software Engineering Matrix

Module	Core Scope	Key Metrics, Methodologies & Formulations
Module I	Process Models & Agile	Waterfall, V-Model, Prototyping, Boehm's Spiral (4 Quadrants), Agile Manifesto, Scrum Roles & Sprint Burndown
Module II	Requirements & Analysis	IEEE 830 SRS Standard, Use Case Modeling, Data Flow Diagrams (DFD Level 0/1/2), Data Dictionary & ER Diagrams
Module III	Design & Architecture	Cohesion (Functional to Coincidental), Coupling (Data to Content), SOLID Principles, GoF Design Patterns
Module IV	Metrics & Estimation	Albrecht Function Points (FP), Basic & Intermediate COCOMO Models, PERT Expected Time & CPM Critical Path
Module V	Testing & Maintenance	Black-Box (ECP, BVA), White-Box (Basis Path & McCabe's Cyclomatic Complexity), Mutation Testing, Maintenance

Exam Preparation & High-Yield Strategy

This publication-grade master book consolidates all 5 modules with step-by-step COCOMO & PERT cost estimation calculations, Cyclomatic Complexity graph proofs, and model answers to BIT Mesra end-semester examination questions.

Module I: Software Process Models & Agile Methodologies

1. The Software Crisis, Engineering Principles & The Dual Nature of Software
2. Generic Software Process Framework (Communication, Planning, Modeling, Construction, Deployment)
3. Classical Waterfall Model: Sequential Phases, Feedback Loops & Major Inherent Flaws
4. The V-Model: Verification vs. Validation Cross-Phase Association
5. Prototyping Paradigm: Throwaway vs. Evolutionary Prototyping Mechanics
6. Boehm's Spiral Model: 4 Quadrants & Explicit Risk-Driven Iterations
7. The Agile Manifesto: 4 Core Values & 12 Guiding Principles
8. The Scrum Framework: Roles (PO, Scrum Master, Team) & Ceremonies
9. Scrum Artifacts: Product Backlog, Sprint Backlog & Sprint Burndown Charts
10. Extreme Programming (XP): Pair Programming, Test-Driven Development (TDD) & CI
11. Process Model Selection Decision Matrix for Diverse Engineering Constraints
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 1 – 4: Traditional Prescriptive Process Models

A **Software Process Model** is an abstract representation of the lifecycle activities, actions, tasks, milestones, and work products required to build high-quality software.

Process Model	Core Operational Philosophy	Ideal Project Characteristics
1. Classical Waterfall	Strictly linear sequential progression. Downstream phases only begin once upstream deliverables are frozen.	Requirements are 100% clear, stable, and well-understood; safety-critical systems with rigid standards.
2. V-Model	Pairs each development phase on the left descending arm with its corresponding testing verification phase on the right ascending arm.	High-reliability systems where test plan generation occurs early in parallel with requirements.
3. Prototyping	Builds a quick mock-up interface to elicit, clarify, and validate ambiguous user requirements.	High user interaction systems where clients cannot articulate exact requirements upfront.
4. Boehm's Spiral Model	Risk-driven iterative model combining iterative prototyping with Waterfall systematic control. Evaluates risk in 4 quadrants per cycle.	Large-scale, expensive, high-risk enterprise systems with emerging requirements.

Topic 6: Boehm's Spiral Model (4 Quadrants of Iteration)

The 4 Quadrants of the Spiral Model Cycle

1. **Quadrant 1: Determine Objectives, Alternatives & Constraints:** Identify specific phase goals, alternative architectural options, and operational constraints (cost, schedule).
2. **Quadrant 2: Identify & Resolve Risks (Risk Analysis):** Detailed risk evaluation; create simulations, benchmarks, or prototypes to mitigate technical and project risks.
3. **Quadrant 3: Develop & Verify Next-Level Product:** Engineering design, coding, and verification testing of the release increment.
4. **Quadrant 4: Review & Plan Next Phase:** Customer evaluation of the increment, review progress, and plan the next spiral loop.

Topic 7 – 9: Agile Methodologies & The Scrum Framework

The 4 Values of the Agile Manifesto (2001): 1. **Individuals and interactions** over processes and tools. 2. **Working software** over comprehensive documentation. 3. **Customer collaboration** over contract negotiation. 4. **Responding to change** over following a rigid plan.

Scrum Component	Definition & Purpose	Key Responsibility / Invariant
Product Owner (PO)	Represents business stakeholders and customers; defines vision.	Owns and prioritizes the Product Backlog.
Scrum Master	Servant-leader; removes organizational impediments and ensures adherence to Scrum.	Facilitates Daily Standups, Sprint Planning, and Retrospectives.
Sprint Backlog	Subset of Product Backlog user stories selected for delivery in current 2–4 week Sprint.	Estimated by developers using Story Points (Planning Poker).
Burndown Chart	Visual graph plotting remaining work (story points) against time (days of the sprint).	Tracks velocity and predicts whether sprint goals will be met on time.

Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Compare the Waterfall Model and the Spiral Model across 4 engineering parameters. (8 Marks)

- 1. Risk Management:** Waterfall has zero explicit risk management mechanisms; Spiral is fundamentally driven by continuous risk analysis in every cycle.
- 2. Handling Changing Requirements:** Waterfall resists changes after requirements freeze; Spiral accommodates evolving requirements in subsequent spiral loops.
- 3. Customer Involvement:** In Waterfall, customer interaction happens only at the beginning (requirements) and end (delivery); in Spiral, customer evaluates prototypes after every loop.
- 4. Project Cost & Complexity:** Waterfall is suitable for small, simple projects; Spiral is designed for large-scale, complex, mission-critical systems.

Module II: Requirements Engineering, Analysis Modeling & Structured Specification

1. Requirements Engineering Process (Elicitation, Analysis, Specification, Validation)
2. Functional Requirements vs. Non-Functional Requirements (FURPS+ Model)
3. IEEE 830 Standard Format for Software Requirements Specification (SRS)
4. Characteristics of High-Quality SRS Documents (Traceable, Unambiguous, Verifiable)
5. Use Case Modeling: Actors, Use Cases, `<>` vs. `<>` Relationships
6. Structured Analysis: Data Flow Diagrams (DFD) — 4 Core Geometric Notations
7. DFD Leveling Hierarchy: Context Diagram (Level 0), Level 1 & Level 2 Explosions
8. DFD Balancing Invariants & Common Modeling Errors (Black Holes, Miracles, Gray Holes)
9. Data Dictionary Architecture: BNF Metadata Syntax & Composite Data Elements
10. Entity-Relationship (ER) Modeling: Entities, Attributes, Cardinalities & Keys
11. Decision Tables & Structured English for Complex Business Logic Modeling
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 2 – 4: Requirements Engineering & The IEEE 830 SRS Standard

A **Software Requirements Specification (SRS)** is a formal contract between the client and the development organization describing the complete functional behavior and operational constraints of the system.

SRS Section (IEEE 830)	Mandatory Content & Description	Key Invariants & Purpose
1. Introduction	Purpose of the system, project scope, definitions, acronyms, and references.	Sets project boundary; prevents scope creep.
2. Overall Description	Product perspective, user classes, operating environment, design constraints, assumptions.	Provides high-level context without dictating low-level implementation details.
3. Specific Requirements	Detailed Functional Requirements, External Interface Requirements (UI, HW, SW), Non-Functional Requirements (Performance, Security, Reliability).	Must be 100% Unambiguous, Complete, Consistent, Verifiable (Testable), and Traceable.

Topic 6 – 8: Data Flow Diagrams (DFD) & Balancing Rules

DFD Symbol	DeMarco & Yourdon	Gane & Sarson	Functional Meaning
Process	Circle (Bubble)	Rounded Rectangle	Transforms input data flows into output data flows.
Data Flow	Directed Arrow	Directed Arrow	Pipeline along which packets of data travel.
Data Store	Two Parallel Lines	Open Rectangle	Repository of data at rest (files, DB tables).
External Entity	Rectangle	Shaded Rectangle	Source or Sink of data outside system boundary.

DFD Balancing & Consistency Invariants

- **Conservation of Data:** A process cannot output data that was not present in or computable from its input data.
- **Black Hole Error:** A process that has inputs but zero outputs.
- **Miracle Error:** A process that generates outputs with zero inputs.
- **Gray Hole Error:** A process where outputs require data not supplied by its inputs.
- **Level Balancing:** All input data flows and output data flows entering/leaving a parent process bubble in DFD Level N must exactly match the external data flows in its child Level $N + 1$ diagram.

Top BIT Mesra Exam Questions & Answers (Module II)

Q1. Differentiate between $\langle \rangle$ and $\langle >$ relationships in Use Case Diagrams with examples. (8 Marks)

1. $\langle \rangle$ (**Mandatory Shared Behavior**): The base use case unconditionally and explicitly invokes the included use case every single time it executes. Used to factor out common repeated subroutines.

Example: 'Withdraw Cash' $\xrightarrow{\langle \rangle}$ 'Authenticate User PIN'.

2. $\langle >$ (**Optional / Conditional Behavior**): The extending use case executes only if a specific extension condition holds true at designated extension points in the base use case. The base use case is completely functional on its own without knowing about the extension.

Example: 'Calculate Premium' $\xleftarrow{\langle >}$ 'Apply Senior Citizen Discount' (*only if* $\text{age} \geq 60$).

Module III: Software Design Engineering, Modularity & Architecture

1. The Software Design Process (Architectural Design, Interface Design, Component Design)
2. Fundamental Design Principles (Abstraction, Information Hiding, Modularity, Refactoring)
3. Module Cohesion Taxonomy: Functional (Best) to Coincidental (Worst) Complete Scale
4. Module Coupling Taxonomy: Data (Best) to Content (Worst) Complete Scale
5. Object-Oriented Design Principles (SOLID: Single Responsibility, Open-Closed, Liskov, Interface, Dependency)
6. Unified Modeling Language (UML): Structural Diagrams (Class, Object, Component)
7. UML Behavioral & Interaction Diagrams: Sequence, State Machine & Activity Diagrams
8. Software Architectural Styles: Layered, Client-Server, Pipe-and-Filter, MVC, Microservices
9. GoF Design Patterns Taxonomy: Creational, Structural & Behavioral Paradigms
10. Creational Patterns: Singleton & Factory Method Implementations & UML
11. Structural & Behavioral Patterns: Adapter, Decorator, Observer & Strategy Patterns
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 3 & 4: Modularity (Cohesion vs. Coupling)

The primary objective of software modularization is to achieve **High Cohesion (Single Responsibility within a module)** and **Low Coupling (Minimal Interdependence between modules)**.

Cohesion Type (High to Low)	Definition & Internal Binding Characteristics	Quality Rating
1. Functional	All elements of a module contribute to executing a single, well-defined mathematical task (e.g., <code>computeSquareRoot()</code>).	★★★★★ (Ideal)
2. Sequential	The output data of one element serves as the direct input to the next element inside the module (e.g., <code>readRecordAndFormat()</code>).	★★★★★ (High)
3. Communicational	Elements operate on the exact same input data structure or produce the same output structure (e.g., <code>updateAndPrintCustomerReport()</code>).	★★★★ (Medium)
4. Procedural	Elements are grouped because they execute in a specific sequential algorithmic order (e.g., <code>readInputsThenValidate()</code>).	★★★ (Fair)
5. Temporal	Elements are grouped simply because they are executed at the same time (e.g., system <code>startupInit()</code> or <code>shutdown()</code>).	★ (Poor)
6. Logical	Elements perform logically similar tasks selected by a control flag (e.g., a single function with a switch case for all math operations).	★ (Poor)
7. Coincidental	Elements have zero meaningful relationship; grouped purely by accident or arbitrary convenience.	✗ (Worst)

Coupling Type (Low to High)	Definition & Inter-Module Communication	Quality Rating
1. Data Coupling	Modules communicate strictly by passing elementary primitive data parameters (e.g., `int`, `float`).	★★★★★ (Ideal)
2. Stamp Coupling	Modules pass complex composite data structures (e.g., entire `CustomerRecord`), even when the callee only needs a few fields.	★★★★★ (Good)
3. Control Coupling	One module passes a control flag (`bool`, `enum`) that dictates the internal execution flow of the other module.	★★★ (Poor)
4. Common Coupling	Multiple modules share and mutate global variables or shared global memory blocks.	★ (Very Poor)
5. Content Coupling	One module directly accesses, modifies, or branches into the internal private code/memory of another module (e.g., modifying private memory).	✗ (Worst)

Topic 8: Software Architectural Styles

Architectural Style	Structural Organization & Communication	Primary Use Cases
1. Layered (N-Tier)	Organized hierarchically into horizontal layers (UI → Business Logic → Data Access → Database). Upper layers invoke lower layers.	Enterprise web applications, OS architectures.
2. Pipe-and-Filter	Data flows through a stream of discrete transformation filters connected by communication pipes.	Unix pipelines, compilers, video processing engines.
3. Model-View-Controller	Decouples business data (Model), user interface presentation (View), and input handling (Controller).	Interactive GUI applications (React, Angular, Django).
4. Microservices	Decomposes system into small, independently deployable services communicating via lightweight REST/gRPC APIs.	Scalable cloud-native enterprise systems (Netflix, Uber).

Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Explain the SOLID design principles in Object-Oriented Software Engineering. (10 Marks)

- **S (Single Responsibility Principle):** A class should have only one reason to change, encapsulating a single responsibility.
- **O (Open/Closed Principle):** Software entities should be open for extension, but closed for modification (achieved via interfaces and polymorphism).
- **L (Liskov Substitution Principle):** Subtypes must be substitutable for their base types without altering program correctness.
- **I (Interface Segregation Principle):** Clients should not be forced to depend on interfaces they do not use (prefer small, client-specific interfaces over fat general ones).
- **D (Dependency Inversion Principle):** High-level modules should not depend on low-level modules; both should depend on abstractions.

Module IV: Software Project Estimation, Metrics & Scheduling

1. Software Project Management Dimensions (People, Product, Process, Project — 4 P's)
2. Size-Oriented Metrics (Lines of Code — LOC) & Inherent Productivity Pitfalls
3. Function-Oriented Metrics: Albrecht's Function Point (FP) Analysis Formulations
4. Unadjusted Function Count (UFC) across 5 Information Domain Characteristics
5. 14 General System Characteristics & Value Adjustment Factor (VAF) Math
6. Boehm's COCOMO I Model: Organic, Semi-Detached & Embedded Project Modes
7. Basic, Intermediate (Cost Drivers / EAF) & Detailed COCOMO Equations
8. Project Scheduling: Work Breakdown Structure (WBS) & Activity Networks
9. Critical Path Method (CPM): Earliest Start/Finish & Latest Start/Finish Slack Times
10. Program Evaluation and Review Technique (PERT) Expected Duration (T_e) & Variance
11. Risk Management Framework: Identification, Projection, Assessment & RMMM Plans
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 3 – 5: Function Point (FP) Analysis

1. Function Point Formula (Albrecht, 1979):

$$FP = UFC \times VAF = UFC \times \left(0.65 + 0.01 \times \sum_{i=1}^{14} F_i \right)$$

Where **UFC** is Unadjusted Function Count, and $F_i \in [0, 5]$ are ratings for 14 General System Characteristics (yielding $VAF \in [0.65, 1.35]$).

Information Domain Element	Low Complexity	Average Complexity	High Complexity
1. External Inputs (EI)	3	4	6
2. External Outputs (EO)	4	5	7
3. External Inquiries (EQ)	3	4	6
4. Internal Logical Files (ILF)	7	10	15
5. External Interface Files (EIF)	5	7	10

Topic 6 & 7: Boehm's COCOMO (Constructive Cost Model)

Basic COCOMO Effort & Schedule Equations:

$$\text{Effort } (E) = a \times (\text{KLOC})^b \quad \text{Person-Months (PM)}$$

$$\text{Development Time } (T_{\text{dev}}) = c \times (E)^d \quad \text{Months}$$

$$\text{Average Staffing (Staff)} = \frac{E}{T_{\text{dev}}} \quad \text{Persons}$$

Project Mode	Characteristics	a	b	c	d
1. Organic	Small teams, familiar in-house domain, relaxed constraints.	2.4	1.05	2.5	0.38
2. Semi-Detached	Medium teams, mixed experience levels, medium constraints.	3.0	1.12	2.5	0.35
3. Embedded	Tight hardware/software constraints, complex safety-critical systems.	3.6	1.20	2.5	0.32

Step-by-Step Solved Problem: Basic COCOMO Estimation for a 30 KLOC Semi-Detached Project

1. Step 1: Compute Effort (E):

$$E = 3.0 \times (30)^{1.12} = 3.0 \times 44.91 = \mathbf{134.73 \text{ Person-Months}}$$

2. Step 2: Compute Development Time (T_{dev}):

$$T_{\text{dev}} = 2.5 \times (134.73)^{0.35} = 2.5 \times 5.54 = \mathbf{13.85 \text{ Months}}$$

3. Step 3: Compute Average Staffing Size:

$$\text{Staff} = \frac{E}{T_{\text{dev}}} = \frac{134.73}{13.85} \approx \mathbf{9.7} \Rightarrow \mathbf{10 \text{ Engineers}}$$

Topic 9 & 10: Project Scheduling (CPM & PERT Calculations)

PERT Three-Point Estimation Formulas: - Optimistic time (a), Most likely time (m), Pessimistic time (b).

$$\text{Expected Duration } (T_e) = \frac{a + 4m + b}{6}$$

$$\text{Standard Deviation } (\sigma) = \frac{b - a}{6}, \quad \text{Variance } (\sigma^2) = \left(\frac{b - a}{6} \right)^2$$

Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. State the components of an RMMM (Risk Mitigation, Monitoring, and Management) plan. (8 Marks)

1. **Risk Mitigation (Proactive Prevention):** Developing actionable strategy to reduce the probability of risk occurrence.

Example: High developer turnover risk → cross-training team members and establishing thorough documentation standards.

2. **Risk Monitoring (Tracking):** Continuously observing warning indicators to detect when a risk is transitioning from potential threat to reality.

Example: Monitoring team morale, overtime hours, and unresolved bug counts.

3. **Risk Management (Contingency Action):** Executing contingency plans when mitigation fails and the risk actually occurs.

Example: Activating backup external contractors if a key developer departs.

Module V: Software Testing Strategies, Quality Assurance & Maintenance

1. Verification vs. Validation (Boehm: "Building product right" vs. "Building right product")
2. Black-Box Testing: Equivalence Class Partitioning (ECP) & Boundary Value Analysis (BVA)
3. White-Box (Structural) Testing: Control Flow Graphs (CFG) & Logic Path Coverage
4. McCabe's Cyclomatic Complexity ($V(G) = E - N + 2P = P + 1$) & Independent Basis Paths
5. Advanced Structural Coverage: Branch, Condition, Multiple Condition & MC/DC
6. Testing Levels Hierarchy: Unit Testing (Stubs & Drivers), Integration & System Testing
7. Integration Strategies: Big-Bang, Top-Down (Depth/Breadth-First) vs. Bottom-Up Sandwich
8. Acceptance Testing (Alpha vs. Beta Testing) & Regression Testing Test-Suite Minimization
9. Mutation Testing: Generating Mutants, Killed Mutants (D) & Mutation Score (MS)
10. Software Quality Assurance: ISO 9126, McCall's Quality Factors & CMMI 5 Levels
11. Software Maintenance: 4 Classical Categories (Corrective, Adaptive, Perfective, Preventive)
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 2: Black-Box (Functional) Testing Techniques

Testing Method	Test Case Derivation Principle	Example for Input Range $[1, 100]$
1. Equivalence Class Partitioning (ECP)	Divides input domain into valid and invalid equivalence classes. One test case is selected to represent each class.	- Valid Class: $[1, 100]$ (e.g., 50) - Invalid Class 1: < 1 (e.g., -5) - Invalid Class 2: > 100 (e.g., 150)
2. Boundary Value Analysis (BVA)	Selects test cases at the extreme boundaries and just inside/outside boundaries ($Min, Min+, Nom, Max-, Max$).	Test values: 1 (Min), 2 (Min+), 50 (Nom), 99 (Max-), 100 (Max), plus invalid boundaries 0 and 101.

Topic 3 & 4: White-Box Testing & McCabe's Cyclomatic Complexity

M McCabe's Cyclomatic Complexity ($V(G)$) Formulas: Let G be the Control Flow Graph of a program: 1. **Formula 1 (Edges & Nodes):**

$$V(G) = E - N + 2P$$

Where E is number of flow graph edges, N is number of nodes, and P is number of connected components ($P = 1$ for single programs). 2. **Formula 2 (Predicate Nodes):**

$$V(G) = P + 1$$

Where P is the number of binary decision/conditional predicate nodes ('if', 'while', 'for'). 3. **Formula 3 (Enclosed Regions):**

$$V(G) = \text{Number of Enclosed Planar Regions} + 1 \text{ (Outer Region)}$$

Step-by-Step Solved Problem: Cyclomatic Complexity & Basis Paths Calculation

Code Snippet:

```
1: if (a > 10) {  
2:     if (b < 20)  
3:         x = 1;  
4:     else  
5:         x = 2;  
6: } else {  
7:     x = 3;  
8: }  
9: return x;
```

1. **Predicate Nodes (P):** Two conditional tests (Node 1: `a > 10`, Node 2: `b < 20`).
2. **Cyclomatic Complexity:** $V(G) = P + 1 = 2 + 1 = 3$.
3. **3 Linearly Independent Basis Paths:** - *Path 1:* 1 → 6 → 7 → 9 (Condition 1 is False). - *Path 2:* 1 → 2 → 3 → 9 (Condition 1 is True, Condition 2 is True). - *Path 3:* 1 → 2 → 5 → 9 (Condition 1 is True, Condition 2 is False).

Topic 9: Mutation Testing & Quality Evaluation

Mutation Score (MS) Formula:

$$\text{Mutation Score (MS)} = \frac{D}{T - E} \times 100\%$$

Where T is total generated mutants, D is number of killed (detected) mutants, and E is number of equivalent mutants (mutants that have identical functional behavior to original program).

Topic 11: Software Maintenance Categories

Maintenance Type	Purpose & Triggering Factor	Typical Effort Share
1. Corrective	Fixing residual latent software defects and bugs discovered in production.	≈ 20%
2. Adaptive	Modifying software to accommodate changes in external environment (new OS, hardware upgrade, legal regulations).	≈ 25%
3. Perfective	Enhancing software functionality, improving performance, and optimizing code maintainability based on user requests.	≈ 50% (Dominant)
4. Preventive	Refactoring and restructuring code to forestall future potential failures (Software Re-engineering).	≈ 5%



Top BIT Mesra Exam Questions & Answers (Module V)

Q1. Compare Stubs and Drivers in Integration Testing with diagrams. (8 Marks)

- **Driver:** A dummy calling program written to invoke and pass test data to the module under test. Used extensively in **Bottom-Up Integration Testing** where high-level parent control modules have not yet been developed.
- **Stub:** A dummy called subroutine that simulates the interface and returns dummy values for sub-modules invoked by the module under test. Used extensively in **Top-Down Integration Testing** where low-level worker modules have not yet been developed.